# Project: Distributed Multi-Model Analytics for E-commerce Data


## IGIRUKWAYO Ildephonse

## ID: 101404


Link: https://github.com/Ildephonse-hub/Distributed_Multi_Model_Analytics_for_E_commerce_Data_IGIRUKWAYO_Ildephonse_101404

# Contents

ABSTRACT ..... 5

1. PROJECT OVERVIEW AND INTRODUCTION ..... 6

1.1 Project Background ..... 6

1.2 Problem Statement ..... 6

1.3 Main Objective ..... 6

1.4 Specific Objectives ..... 6

1.5 Key Learning Objectives ..... 7

1.6 Project Scope ..... 7

2. SYSTEM ARCHITECTURE AND DEVELOPMENT ENVIRONMENT ..... 7

2.1 System Architecture ..... 7

2.2 Technologies Used ..... 10

2.3 Development Environment ..... 10

PART 1: DATA MODELING AND STORAGE (25%) ..... 11

3. DATASET GENERATION AND PREPARATION ..... 11

3.1 Dataset Description ..... 11

3.2 Dataset Configuration ..... 11

3.3 Dataset Record Counts ..... 11

3.4 Data Relationships ..... 11

3.5 Data Validation ..... 12

4. MONGODB IMPLEMENTATION ..... 13

4.1 Introduction ..... 13

4.2 MongoDB Database and Collections ..... 13

4.3 MongoDB Data Modeling Decisions ..... 14

4.4 Why MongoDB Was Selected ..... 15

4.5 MongoDB Indexing ..... 15

4.6 MongoDB Analytical Queries ..... 16

5. APACHE HBASE IMPLEMENTATION ..... 17

5.1 Introduction ..... 17
5.2 HBase Data Modeling Decisions ..... 17
5.3 Why HBase Was Selected ..... 19
5.4 HBase Implementation ..... 19
5.5 HBase Data Import and Verification ..... 20
5.6 MongoDB and HBase Comparison ..... 20
5.7 Technology Selection Justification ..... 20
PART 2: DATA PROCESSING WITH APACHE SPARK (30%) ..... 21
6. APACHE SPARK IMPLEMENTATION ..... 21
6.1 Introduction ..... 21
6.2 Spark Processing Pipeline ..... 21
6.3 Data Cleaning and Normalization ..... 22
6.4 Spark DataFrame Operations ..... 22
6.5 Spark SQL Analytics ..... 23
6.6 Cohort Analysis ..... 23
6.7 Product Recommendation Indicators ..... 23
6.8 Batch Processing ..... 24
6.9 Parquet Output ..... 24
6.10 Spark Resource Configuration ..... 24
6.11 Why Spark Is Useful for Big Data ..... 25
PART 3: ANALYTICS INTEGRATION (25%) ..... 26
7. INTEGRATED ANALYTICAL DESIGN ..... 26
7.1 Introduction ..... 26
7.2 Integrated Query 1: Customer Value and Engagement Analysis ..... 26
7.3 Integrated Query 2: Product View-to-Purchase Analysis ..... 27
7.4 Integrated Query 3: Conversion Funnel Analysis ..... 28
7.5 Integration Architecture ..... 29
7.6 Performance Considerations ..... 29
PART 4: VISUALIZATION AND INSIGHTS (20%) ..... 30

8. DATA VISUALIZATION AND BUSINESS ANALYSIS ..... 30
8.1 Introduction ..... 30
8.2 Customer Segmentation ..... 30
8.3 Product Performance: Most Viewed Products ..... 31
8.4 Product Performance: Most Purchased Products ..... 32
8.5 Conversion Funnel Analysis ..... 33
8.6 Daily Sales Trends ..... 34
8.7 Summary of Visualization Findings ..... 35
9. SCALABILITY AND TECHNOLOGY SELECTION ..... 36
**9.1 MongoDB Scalability** ..... 36
9.2 HBase Scalability ..... 36
9.3 Spark Scalability ..... 36
9.4 Technology Selection Summary ..... 36
10. CHALLENGES AND LIMITATIONS ..... 37
10.1 Hardware and Resource Constraints ..... 37
10.2 Synthetic Data Limitations ..... 37
12. CONCLUSION ..... 37
Appendix ..... 39
List of products Categories Name ..... 39

# ABSTRACT

The rapid growth of e-commerce platforms generates large volumes of data from customers, products, transactions, browsing sessions, and inventory activities. Managing and analyzing thes datasets requires appropriate data storage models and processing technologies.

This project designs and implements a Big Data analytics solution for an e-commerce platform using three complementary technologies: MongoDB, Apache HBase, and Apache Spark. MongoDB provides document-oriented storage for flexible e-commerce records, Apache HBase introduces a wide-column storage approach for large-scale and time-series data, while Apache Spark provides data processing and analytical capabilities.

A synthetic dataset was generated to simulate 90 days of e-commerce activities. It contains 10,000 users, 5,000 products, 25 categories, 500,000 transactions, and 2,000,000 browsing sessions, giving a total of 2,515,025 records.

MongoDB was used for document storage, validation, indexing, and analytical queries. HBase was configured to demonstrate wide-column storage, table operations, and sample record access. Spark was applied to data cleaning, DataFrame operations, SQL analytics, cohort analysis, product recommendation indicators, and batch processing.

The analytical findings show approximately 466.58 million in total transaction revenue. Customer segmentation identified that the top 25% of customers generated approximately 30.57% of total revenue. Product analysis showed that the most viewed product was different from the most purchased product.

# 1. PROJECT OVERVIEW AND INTRODUCTION

## 1.1 Project Background

E-commerce platforms continuously generate data through customer registration, product browsing, shopping cart activities, purchases, payments, and inventory management.

These activities produce different types of information. Customer profiles and product descriptions are naturally represented as documents, while browsing events and product performance measurements can be represented using wide-column storage. Large-scale analytical operations require processing engines capable of handling significant data volumes.

This project applies MongoDB, Apache HBase, and Apache Spark to demonstrate how different Big Data technologies can be selected and combined according to their strengths.

The project focuses not only on storing information but also on understanding customer behavior, identifying business opportunities, and generating meaningful analytical results.

## 1.2 Problem Statement

E-commerce businesses face challenges when managing large volumes of interconnected customer, product, transaction, and browsing information.

Traditional data management approaches may become inefficient when the volume and complexity of data increase. Businesses need storage systems that support different data access patterns and processing technologies that can analyze large datasets efficiently.

This project addresses these challenges through the strategic application of MongoDB, HBase, and Spark.

## 1.3 Main Objective

To design and implement a multi-faceted Big Data analytics system for e-commerce using MongoDB, Apache HBase, and Apache Spark.

## 1.4 Specific Objectives

1. Generate and prepare a synthetic e-commerce dataset.
2. Design and implement appropriate MongoDB document structures.
3. Design and implement HBase wide-column storage structures.
4. Apply database ingestion, querying, and validation techniques.
5. Use Spark to clean, transform, and analyze large datasets.
6. Demonstrate analytical workflows involving different data sources.

7. Generate meaningful business insights from customer, product, transaction, and session data.
8. Communicate analytical findings through visualizations.

### 1.5 Key Learning Objectives

The project addresses the following learning objectives:

- Understand the trade-offs between document-oriented and wide-column NoSQL databases.
- Design data schemas based on query patterns.
- Implement data ingestion and querying in MongoDB and HBase.
- Develop Spark batch processing applications.
- Integrate information from different data stores and datasets.
- Present analytical results in a meaningful business context.

### 1.6 Project Scope

The project covers five main e-commerce entities:

- Users
- Products
- Categories
- Transactions
- Browsing sessions

The analysis includes customer spending, product performance, transaction revenue, browsing behavior, conversion patterns, and sales trends.

# 2. SYSTEM ARCHITECTURE AND DEVELOPMENT ENVIRONMENT

### 2.1 System Architecture

The solution uses three main technology layers.

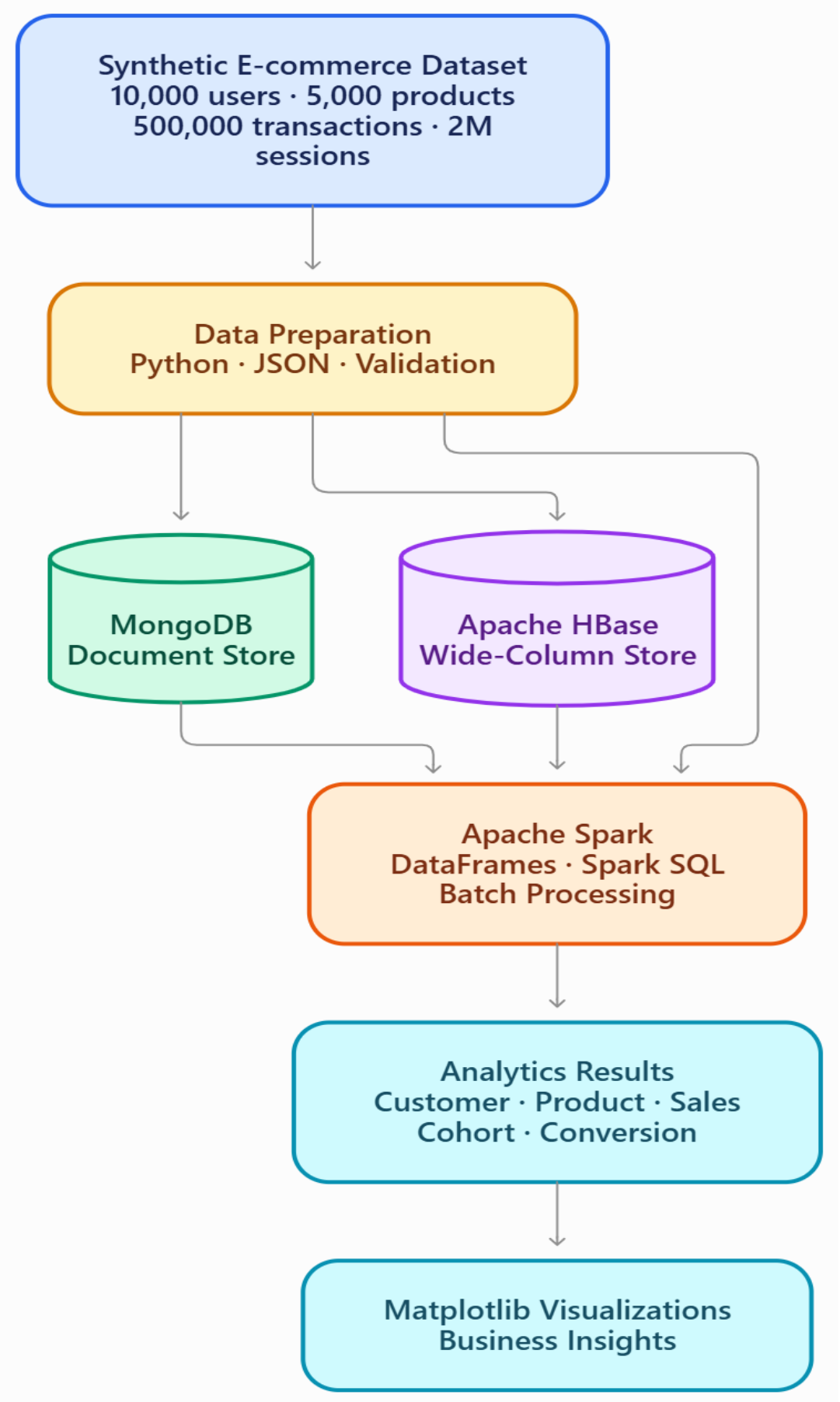
**Storage layer:** MongoDB stores document-oriented e-commerce data, while HBase provides a wide-column storage model for large-scale records.

**Processing layer:** Apache Spark processes exported datasets using DataFrames, SQL operations, transformations, aggregations, and batch processing.

**Analytics and presentation layer:** Analytical outputs are prepared for business interpretation and visualization.

The overall architecture is illustrated below.

Big Data E-commerce System Architecture


Synthetic E-commerce Dataset
10,000 users · 5,000 products
500,000 transactions · 2M sessions
Data Preparation
Python · JSON · Validation
MongoDB
Document Store
Apache HBase
Wide-Column Store
Apache Spark
DataFrames · Spark SQL
Batch Processing
Analytics Results
Customer · Product · Sales
Cohort · Conversion
Matplotlib Visualizations
Business Insights

The intended data flow is:

1. Generate the synthetic e-commerce dataset.
2. Prepare and validate the generated JSON files.
3. Load the relevant data into MongoDB.
4. Prepare and load representative data into HBase.
5. Read datasets and analytical exports into Spark.
6. Perform transformations, joins, aggregations, and analytics.
7. Save analytical outputs.
8. Generate visualizations and business recommendations.

## 2.2 Technologies Used

| Technology | Purpose |
|---|---|
| **MongoDB** | Document-oriented data storage and querying |
| **Apache HBase** | Wide-column data storage |
| **Apache Spark** | Distributed data processing and analytics |
| **Python** | Dataset generation and supporting scripts |
| **PySpark** | Spark applications |
| **Java** | Runtime environment for Spark and Hadoop-related components |
| **Docker** | HBase deployment environment |
| **Matplotlib** | Static data visualization |
| **Windows 11** | Development operating system |

## 2.3 Development Environment

The implementation used the following environment:

| Component | Version or configuration |
|---|---|
| **Operating system** | Windows 11 Pro |
| **Python** | 3.12 |
| **Apache Spark** | 3.5.9 |
| **Scala** | 2.12.18 |
| **Java** | JDK 8 environment |
| **MongoDB** | 8.3.1 |
| **Apache HBase** | 2.6.7 environment |
| **Spark execution** | local[2] |
| **Project directory** | D:\Individual_Final_Project_ild |

Spark was configured to use two local processing threads. This was a resource-conscious choice for the available development computer. It demonstrates Spark application development but does not represent a multi-machine distributed cluster benchmark.

# PART 1: DATA MODELING AND STORAGE (25%)

## 3. DATASET GENERATION AND PREPARATION

### 3.1 Dataset Description

The project used the provided Python dataset generator to create a synthetic e-commerce dataset.

The Faker and pandas libraries were used in the data-generation environment.

The generated data simulates customer and business activities over 90 days.

### 3.2 Dataset Configuration

| Dataset parameter | Configured value |
|---|---|
| **Users** | 10,000 |
| **Products** | 5,000 |
| **Categories** | 25 |
| **Transactions** | 500,000 |
| **Sessions** | 2,000,000 |
| **Duration** | 90 days |
| **Random seed** | 42 |

### 3.3 Dataset Record Counts

The generated dataset contained:

| Dataset | Records |
|---|---|
| **Users** | 10,000 |
| **Products** | 5,000 |
| **Categories** | 25 |
| **Transactions** | 500,000 |
| **Sessions** | 2,000,000 |
| Total | **2,515,025** |

### 3.4 Data Relationships

The dataset contains several important relationships.

**User–Session–Transaction Relationship**

A user can have multiple browsing sessions. Each session belongs to one user and may result in a transaction.

A transaction may reference the session that generated it.

**Product– Category Relationship**

Each product belongs to a category and may also belong to a subcategory.

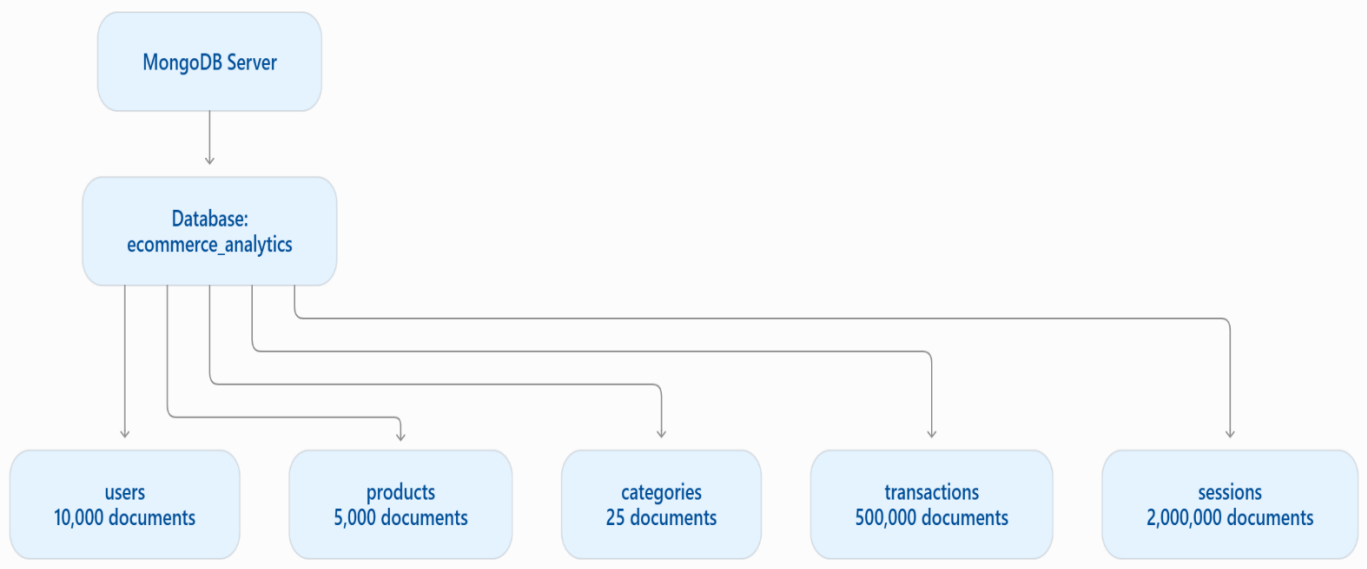
**Session–Product–Cart–Transaction Relationship**

During a session, users can view products, add products to their carts, and purchase products.

**Inventory Relationship**

Products contain stock information. Stock availability affects whether products can be purchased.

Figure 2: E-commerce Dataset Entity Relationship Diagram




## 3.5 Data Validation

Data validation was performed to establish the reliability of the generated dataset.

The validation process checked:

- Required fields.
- Unique transaction identifiers.
- User references.
- Product references.
- Category references.
- Financial calculations.
- Timestamp availability.
- Data completeness.

The transaction validation confirmed 500,000 transaction records and reported no duplicate transaction IDs or missing required transaction identifiers, user references, timestamps, financial values, or item information.

Financial validation also reported zero mismatches in the transaction calculations.

The user, product, category, and session relationships were checked during dataset validation.

# 4. MONGODB IMPLEMENTATION

## 4.1 Introduction

MongoDB is a document-oriented NoSQL database that stores information in BSON documents.

It is suitable for e-commerce data because the data contains nested structures and relationships that can be represented naturally inside documents.

MongoDB was selected for customer profiles, product information, categories, transactions, and browsing sessions.

## 4.2 MongoDB Database and Collections

The database created for the project was:

ecommerce analytics

It contains five main collections:

1. users
2. products
3. categories
4. transactions
5. sessions

The collection record counts were verified as follows:

| Collection | Documents |
|---|---|
| **users** | 10,000 |
| **products** | 5,000 |
| **categories** | 25 |
| **transactions** | 500,000 |
| **sessions** | 2,000,000 |

**N.B:** The code of product is from `prod_00000` to `prod_04999`.

## 4.3 MongoDB Data Modeling Decisions

### *A. User Profile Model*

User information is stored as documents containing user identifiers and profile attributes.

This model allows flexible storage of demographic and account information.

A user document can also support embedded information such as preferences or summarized purchase history when those fields are required frequently.

### *B. Product Catalog Model*

Products are stored as documents containing product identifiers, descriptions, prices, stock information, category identifiers, and subcategory identifiers.

Category information is maintained separately to avoid unnecessary duplication of category records.

### *C. Transaction Model*

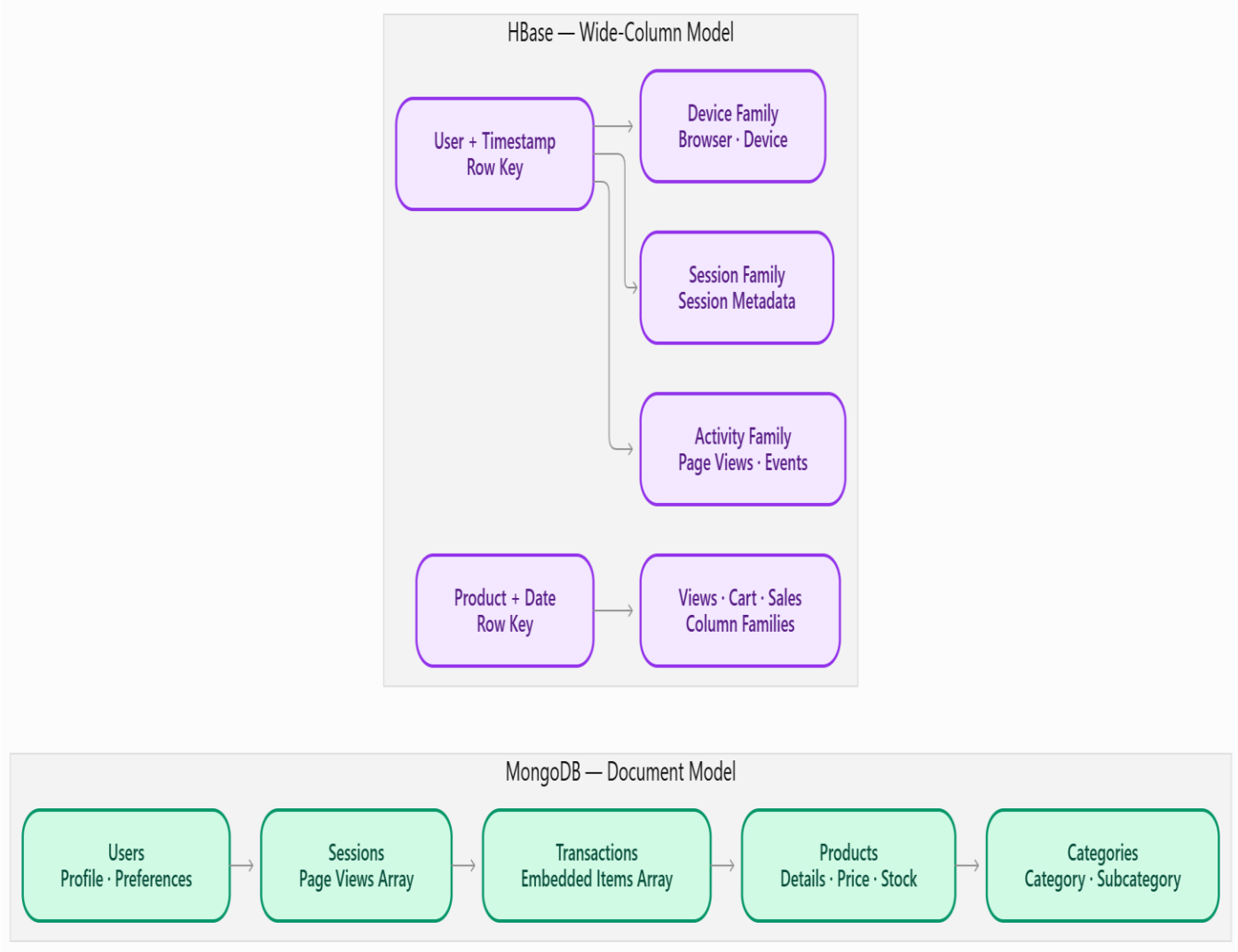
Transactions contain transaction-level information and an embedded items array.

Embedding line items is appropriate because the items belong directly to a transaction and are normally retrieved together with transaction information.

### *D. Session Model*

Session documents represent browsing activities and can contain page-view information in arrays.

This structure supports analysis of the customer's journey within a session.




## 4.4 Why MongoDB Was Selected

MongoDB was selected because:

- It supports flexible document structures.
- It supports nested objects and arrays.
- It allows transaction line items to be embedded.
- It supports aggregation pipelines.
- It provides indexes for efficient retrieval.
- It supports queries over related fields and nested structures.

The document model is especially useful when the application frequently retrieves complete customer, product, transaction, or session records.

## 4.5 MongoDB Indexing

Indexes were created for frequently queried identifiers and analytical fields.

The implementation included indexes for:

- Product identifiers.
- User identifiers.
- Category identifiers.
- Session identifiers.
- Transaction identifiers.
- User references in transactions and sessions.
- Transaction timestamps.

The indexes were examined using MongoDB query execution plans to understand their contribution to query performance.

## 4.6 MongoDB Analytical Queries

MongoDB aggregation pipelines were used to extract useful business information.

### *Query 1: Customer Spending Analysis*

**Business question:** Which customers generate the highest transaction value?

The aggregation groups transactions by user_id, calculates total spending and average transaction value, sorts customers by total spending, and returns the highest-ranking customers.

A verified result was:

| Metric | Result |
|---|---|
| **Customer** | user_005142 |
| **Transactions** | 73 |
| **Total spending** | 80,881.92 |
| **Average transaction value** | 1,107.97 |

This analysis demonstrates how MongoDB aggregation can identify high-value customers.

### *Query 2: Product Sales Analysis*

**Business question:** Which products generate the highest sales quantities?

The transaction item arrays can be unwound to obtain individual product purchases. The resulting records can be grouped by product identifier and summarized by units sold and purchase records.

The highest-selling product by quantity was:

| Metric | Result |
|---|---|
| **Product ID** | prod_00283 |
| **Units sold** | 661 |
| **Purchase records** | 323 |

*Query 3: Category Revenue Analysis*

**Business question:** Which product categories generate the highest revenue?

The category analysis groups product purchase information by category and calculates total revenue and purchase quantities.

The highest-revenue category was:

| Metric | Result |
|---|---|
| **Category ID** | cat_005 |
| **Category name** | Cameron-Parsons |
| **Revenue** | 22,221,952.21 |

These analyses demonstrate MongoDB's ability to perform aggregation over e-commerce documents and nested transaction items.

# 5. APACHE HBASE IMPLEMENTATION

## 5.1 Introduction

Apache HBase is a distributed wide-column NoSQL database built for large-scale data storage and access.

Unlike MongoDB, which stores flexible documents, HBase organizes data into tables, rows, column families, and columns.

HBase was selected to demonstrate an alternative model for high-volume browsing and time-series information.

## 5.2 HBase Data Modeling Decisions

The project considered two major data categories.

### *A. User Browsing and Session Data*

Browsing activity is time-dependent and can generate many records for individual users.

A suitable HBase row-key design is:

user_id + timestamp

This structure supports retrieving records belonging to a particular user and scanning their activity over a time range.

For example:

user_000042#20261001103000

The proposed column families are:

- session: session identifiers and session metadata.
- activity: page type, product viewed, and event details.
- device: device and browser information.

### *B. Product Performance Data*

Product performance metrics can be organized by product and date.

A suitable row-key design is:

product_id + date

For example:

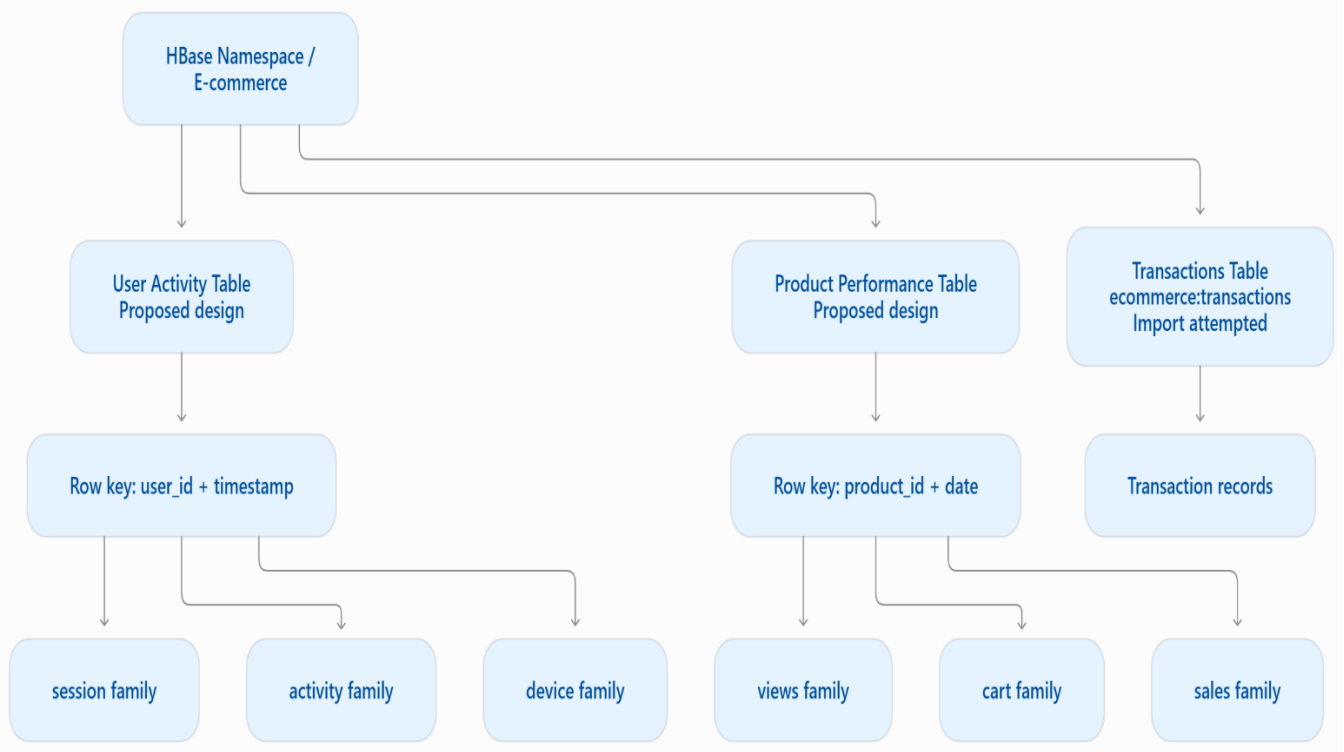
prod_00123#2026-10-01

Possible column families include:

- views: product view counts.
- cart: cart addition counts.
- sales: units sold and revenue.

These row-key patterns are design choices based on the expected query patterns. They are presented as the logical HBase schema design, not as a claim that every example row key or family was individually verified in the deployed tables.

Figure 4: HBase Wide-Column Data Model




## 5.3 Why HBase Was Selected

HBase provides several advantages for large-scale e-commerce data:

- Efficient row-key-based retrieval.
- Support for sparse data.
- Column-family organization.
- Time-series data access.
- Horizontal scalability through distributed storage.
- Suitable access patterns for high-volume event records.

For example, retrieving a user's browsing records using a user identifier and timestamp range is a natural HBase use case.

## 5.4 HBase Implementation

The HBase environment was deployed using Docker.

The implementation involved:

1. Starting the HBase environment.

2. Accessing the HBase shell.
3. Preparing e-commerce data for loading.
4. Creating and inspecting HBase tables.
5. Executing sample record retrieval operations.
6. Checking the stored records.

## 5.5 HBase Data Import and Verification

The transaction import process targeted the HBase table:

ecommerce: transactions

The planned import covered 500,000 transaction records using batches of 500.

## 5.6 MongoDB and HBase Comparison

| Feature | MongoDB | Apache HBase |
|---|---|---|
| **Data model** | Document | Wide-column |
| **Data organization** | Collections and BSON documents | Tables, rows, column families |
| **Schema** | Flexible | Column-family structure |
| **Data access** | Document queries and aggregation | Row-key access and scans |
| **Nested data** | Native support | Requires column organization |
| **Time-series access** | Possible through documents and indexes | Suitable for row-key-based time-range access |
| **Main project role** | Customer, product, transaction, and session documents | Large-scale event and time-series storage design |

## 5.7 Technology Selection Justification

MongoDB is appropriate when the application needs flexible records, nested structures, and rich aggregation operations.

HBase is appropriate when the application needs predictable row-key-based access to large volumes of sparse or time-series records.

The two databases are therefore complementary rather than direct replacements for each other.

# PART 2: DATA PROCESSING WITH APACHE SPARK (30%)

## 6. APACHE SPARK IMPLEMENTATION

### 6.1 Introduction

Apache Spark was used to process and analyze the generated e-commerce datasets.

Spark provides DataFrame operations, SQL functionality, and batch processing capabilities that are useful for transforming large datasets.

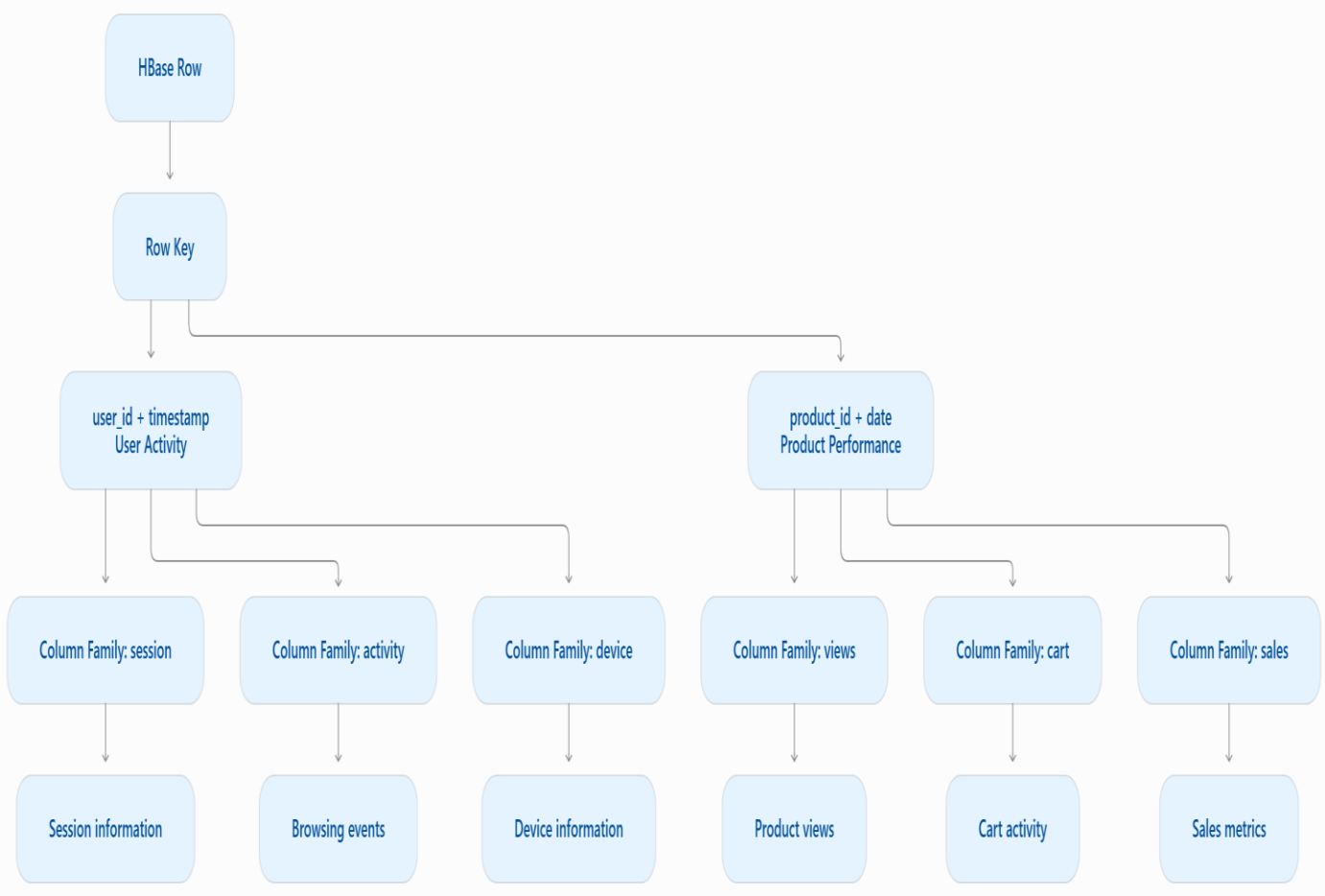
The project used PySpark with Spark 3.5.9.

### 6.2 Spark Processing Pipeline

The processing workflow consisted of:

1. Reading generated JSON datasets.
2. Inspecting schemas and record counts.
3. Cleaning and preparing analytical fields.
4. Transforming and aggregating data.
5. Performing DataFrame operations.
6. Executing Spark SQL queries.
7. Preparing cohort and recommendation indicators.
8. Saving analytical outputs in suitable formats.

Figure 5: Apache Spark Data Processing Pipeline




## 6.3 Data Cleaning and Normalization

The first stage involved reading the generated JSON data and examining its structure.

The cleaning and preparation work included:

- Inspecting schemas.
- Checking record counts.
- Examining missing and inconsistent fields.
- Preparing numerical and categorical fields.
- Standardizing data for analytical operations.
- Selecting relevant fields for processing.

These activities helped prepare the data for reliable aggregation and analysis.

## 6.4 Spark DataFrame Operations

Spark DataFrames were used to read and transform the JSON datasets.

Operations included:

- Selecting relevant columns.
- Filtering records.
- Grouping records.
- Aggregating numerical values.
- Joining related datasets.
- Inspecting schemas and output records.

A DataFrame join was performed to combine product and category-related information for analytical purposes.

## 6.5 Spark SQL Analytics

Spark SQL was used to perform queries on DataFrames created from the generated datasets.

The SQL operations included grouping, filtering, aggregation, and joining.

The analysis covered:

- Daily sales revenue.
- Product purchase indicators.
- Category-level revenue.
- Session conversion statuses.
- Browsing behavior.
- Conversion funnel stages.

Spark SQL provides a familiar query language for analytical processing while using Spark's execution engine.

## 6.6 Cohort Analysis

Cohort analysis groups customers according to a shared characteristic, such as registration period, and examines their behavior over time.

For an e-commerce platform, cohort analysis can help answer questions such as:

- Do customers registered in the same month show similar purchasing behavior?
- How does customer spending change after registration?
- Which customer groups remain active over time?

The project included cohort analysis in its Spark processing work to demonstrate the use of grouped customer behavior for retention analysis.

## 6.7 Product Recommendation Indicators

Product recommendation indicators can be generated using purchase and browsing behavior.

Two useful approaches are:

**Frequently purchased together:** Identify products that appear together in transaction item lists.

**Frequently viewed together:** Identify products that appear in the same browsing sessions.

These indicators can support recommendations by identifying relationships between products.

The project prepared product performance indicators based on views and purchases, which provide useful inputs for recommendation strategies.

## 6.8 Batch Processing

Batch processing was used to process e-commerce datasets and generate analytical results.

Rather than processing only individual records, Spark processes groups of records through transformations and aggregations.

This approach is useful when analyzing hundreds of thousands of transactions and millions of browsing sessions.

## 6.9 Parquet Output

The project saved analytical output in Parquet format.

One verified output contained 25 category records with the following schema:

| Column | Data type |
|---|---|
| **category_id** | string |
| **purchase_records** | long |
| **total_revenue** | double |

The saved output was read again for verification.

The highest-revenue category was cat_005, with revenue of 22,221,952.21.

Parquet is useful for analytical workloads because it stores data in a columnar format and supports efficient reading of selected columns.

## 6.10 Spark Resource Configuration

The project used local [2] execution to limit resource consumption.

This was appropriate for the available development environment.

The configuration enabled practical Spark development and batch analytics without claiming that the project was executed on a multi-node cluster.

## 6.11 Why Spark Is Useful for Big Data

As e-commerce datasets grow, analytical operations become more demanding.

Spark offers:

- Distributed processing capabilities.
- In-memory computation.
- DataFrame and SQL APIs.
- Parallel execution.
- Support for large-scale batch processing.
- Integration with different data formats and storage systems.

These capabilities make Spark useful for processing growing e-commerce datasets and generating analytical results.

# PART 3: ANALYTICS INTEGRATION (25%)

# 7. INTEGRATED ANALYTICAL DESIGN

## 7.1 Introduction

The purpose of analytics integration is to demonstrate how information from different storage systems can be combined to answer business questions that cannot be answered efficiently by examining only one dataset.

MongoDB contains customer, product, and transaction information. HBase is designed to store large-scale browsing and time-series events. Spark provides the processing engine for combining and analyzing data from different sources.

The project uses the available MongoDB data, generated JSON datasets, and Spark analytical outputs to establish an integration workflow.

A direct live MongoDB–HBase connector and a complete cross-database Spark join were not confirmed as executed. Therefore, the integration described in this section distinguishes the implemented analytical datasets from the proposed full connector-based workflow.

## 7.2 Integrated Query 1: Customer Value and Engagement Analysis

### Business Question

Which customers generate the highest revenue, and how can their browsing activity help explain their purchasing behavior?

### Data Sources

| Data source | Information |
|---|---|
| **MongoDB users** | Customer profile information |
| **MongoDB transactions** | Transaction history and spending |
| **HBase sessions** | Browsing activity and session events |
| **Apache Spark** | Data combination and analytical processing |

### Processing Workflow

1. Retrieve customer profiles from MongoDB.
2. Retrieve transaction history and calculate spending per user.
3. Retrieve session activity from HBase using the user identifier.

4. Load the extracted datasets into Spark DataFrames.
5. Join the datasets using user_id.
6. Calculate total spending, number of transactions, and browsing activity.
7. Group customers according to their value and engagement.

**Expected Analytical Output**

| Customer metric | Description |
|---|---|
| **Total spending** | Sum of customer transaction values |
| **Transaction count** | Number of transactions per customer |
| **Session count** | Number of browsing sessions |
| **Average session duration** | Average time spent browsing |
| **Customer segment** | Spending-based customer group |

**Business Value**

This analysis can help identify customers who generate high revenue and customers who show strong engagement but have not yet converted.

The business can use these findings to create personalized promotions and retention strategies.

## 7.3 Integrated Query 2: Product View-to-Purchase Analysis

**Business Question**

Which products receive high customer attention, and which products successfully convert that attention into purchases?

**Data Sources**

- MongoDB product catalog.
- MongoDB transaction records.
- HBase product-view and interaction records.
- Spark for joining and aggregation.

**Processing Workflow**

1. Retrieve product information from MongoDB.
2. Retrieve product view information from HBase.
3. Retrieve purchase information from MongoDB.
4. Load the datasets into Spark DataFrames.
5. Join the datasets using product_id.
6. Calculate views, purchase records, and units sold.
7. Rank products according to views and purchases.
8. Identify products with high views but comparatively low purchases.

### Business Value

This analysis can help identify products that attract attention but do not convert into purchases.

Businesses can improve product descriptions, images, pricing, or promotional strategies to increase sales.

## 7.4 Integrated Query 3: Conversion Funnel Analysis

### Business Question

At which stage of the customer journey do customers abandon the purchasing process?

### Data Sources

| System | Information |
|---|---|
| **HBase** | Browsing events and page-view history |
| **MongoDB** | Transaction and purchase information |
| **Spark** | Sequential event analysis and aggregation |

### Processing Workflow

1. Retrieve browsing events in chronological order.
2. Identify the main customer journey stages.
3. Match browsing activity to the appropriate session.
4. Identify sessions that progress from home to product detail, cart, checkout, and confirmation.
5. Match confirmed purchases with transaction records.
6. Calculate the number of sessions at each stage.
7. Calculate stage-to-stage drop-off percentages.

### Implemented Analytical Result

The project performed sequential conversion funnel analysis using the available session data.

| Funnel stage | Sessions |
|---|---|
| **Home** | 1,548,376 |
| **Product detail** | 821,381 |
| **Cart** | 536,727 |
| **Checkout** | 333,762 |
| **Confirmation** | 245,010 |

The largest drop occurred between the home page and product detail stage, with a drop-off rate of approximately 46.95%.

The overall home-to-confirmation conversion rate was approximately 15.83%.

**Business Value**

The result indicates that product discovery and navigation should be improved to encourage more customers to proceed from the home page to product pages.

## 7.5 Integration Architecture

The integration workflow is illustrated below.

[Insert Figure 6: MongoDB–HBase–Spark Analytics Integration]

MongoDB and HBase provide different storage capabilities, while Spark provides a common processing environment.

For a full production implementation, the data can be extracted through database connectors or exported into compatible files before being loaded into Spark.

## 7.6 Performance Considerations

Several considerations are important when integrating the systems:

- Use indexes in MongoDB for frequently queried fields.
- Use well-designed HBase row keys for efficient record retrieval.
- Filter unnecessary data before loading it into Spark.
- Select only required columns.
- Use appropriate partitioning for large datasets.
- Avoid unnecessary joins and data shuffling.
- Store analytical outputs in efficient formats such as Parquet.

The local Spark execution environment was sufficient for project development, but a larger deployment would benefit from a multi-node Spark cluster and a properly configured distributed HBase environment.

# PART 4: VISUALIZATION AND INSIGHTS (20%)

## 8. DATA VISUALIZATION AND BUSINESS ANALYSIS

### 8.1 Introduction

Visualization transforms analytical results into graphical information that can be understood by business users.

The project generated five static graphs using Matplotlib.

The graphs address customer segmentation, product performance, conversion funnel behavior, and daily sales trends.

The graph files are stored in:

D:\Individual_Final_Project_ild\visualization_graphs

### 8.2 Customer Segmentation

**Business question:** Which customer groups generate the most revenue?

Customers were divided into four spending segments.

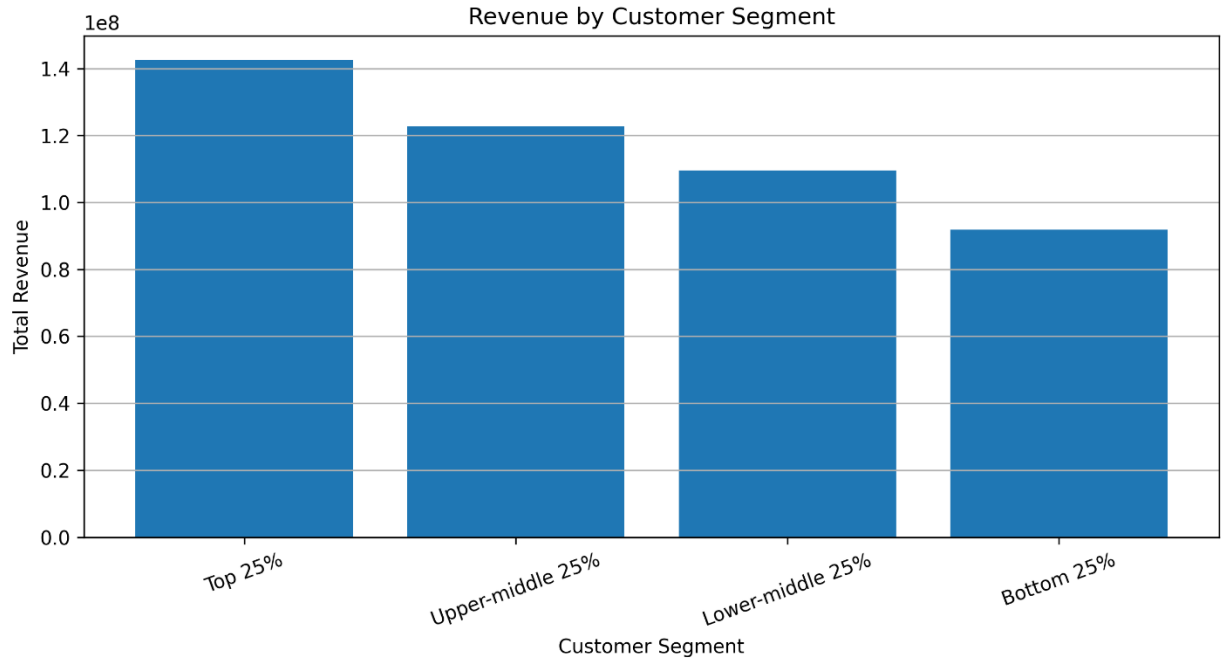
| Segment | Customers | Revenue |
|---|---|---|
| **Top 25%** | 2,500 | 142,615,340.24 |
| **Upper-middle 25%** | 2,500 | 122,643,244.86 |
| **Lower-middle 25%** | 2,500 | 109,523,966.56 |
| **Bottom 25%** | 2,500 | 91,793,611.56 |

The top 25% generated approximately 30.57% of total revenue.

**Interpretation:** The highest-spending customer group contributes the largest revenue share.

**Recommendation:** Introduce customer loyalty programs, personalized offers, and retention strategies for high-value customers.

Customer Segmentation customer segments




## 8.3 Product Performance: Most Viewed Products

**Business question:** Which products attract the most customer attention?

The analysis ranked products according to their number of recorded views.

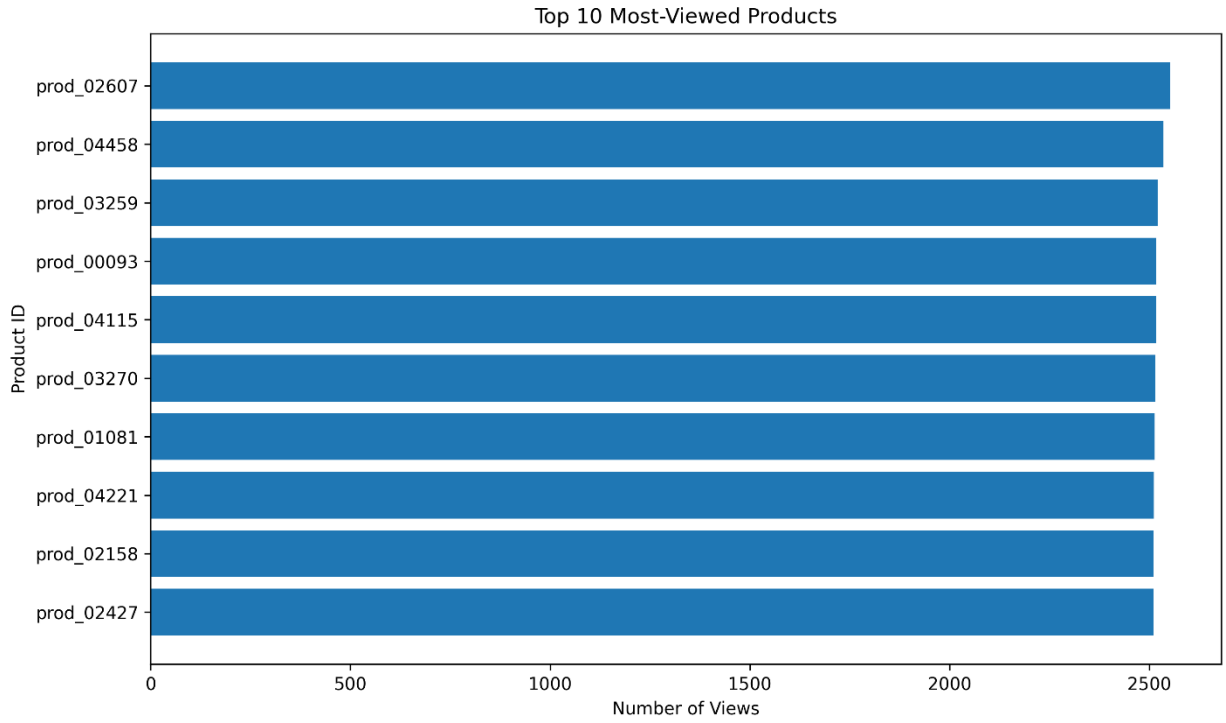
The most viewed product was:

- Product ID: prod_02607
- Number of views: 2,553

**Interpretation:** Product views provide an indication of customer interest and product visibility.

**Recommendation:** Monitor highly viewed products and examine whether their views result in purchases.

Top Viewed Products top viewed products




## 8.4 Product Performance: Most Purchased Products

The analysis also ranked products according to their sales quantities.

The most purchased product was:

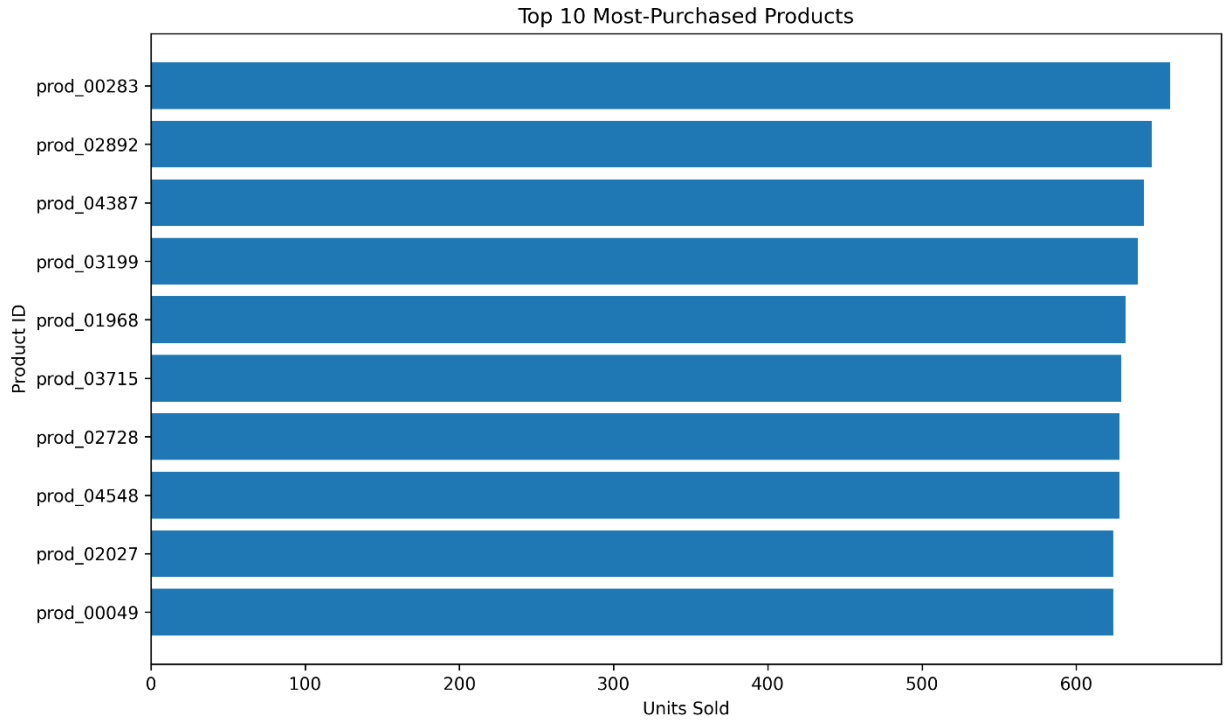
- Product ID: prod_00283
- Units sold: 661
- Purchase records: 323

**Interpretation:** Product popularity based on views differs from popularity based on purchases.

This demonstrates that customer attention does not automatically result in a completed purchase.

**Recommendation:** Compare viewed products with purchased products to identify opportunities for improving pricing, descriptions, images, and promotions.

Top Purchased Products top purchased products.




## 8.5 Conversion Funnel Analysis

**Business question:** At which stage do customers abandon the purchasing process?

| Stage | Sessions | Drop-off |
|---|---|---|
| **Home** | 1,548,376 | |
| **Product detail** | 821,381 | 46.95% |
| **Cart** | 536,727 | 34.66% |
| **Checkout** | 333,762 | 37.81% |
| **Confirmation** | 245,010 | 26.59% |

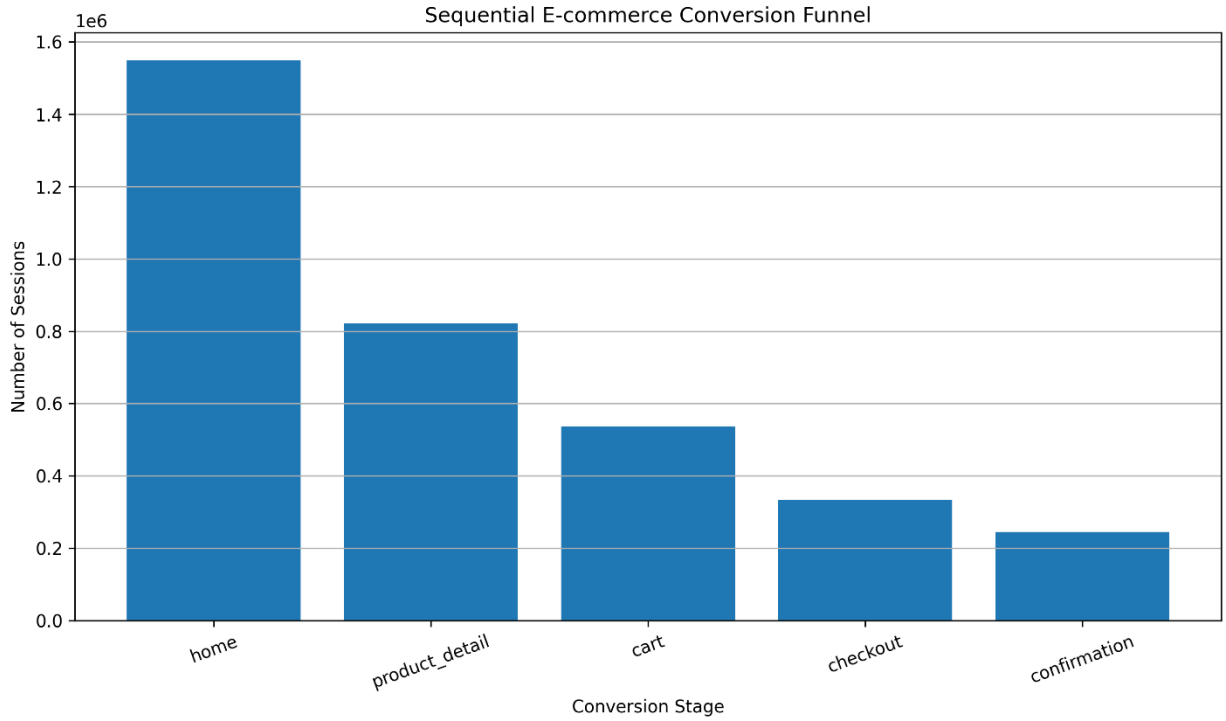
The largest drop-off occurred between the home page and product detail stage.

The overall conversion from home to confirmation was approximately 15.83%.

**Interpretation:** A significant number of sessions did not progress from the home page to product details.

**Recommendation:** Improve homepage product discovery, search functionality, category navigation, and product recommendations.

Conversion Funnel conversion funnel.




## 8.6 Daily Sales Trends

**Business question:** How does revenue change over time?

The sales analysis covered 91 dates from 31 July to 29 October 2026.

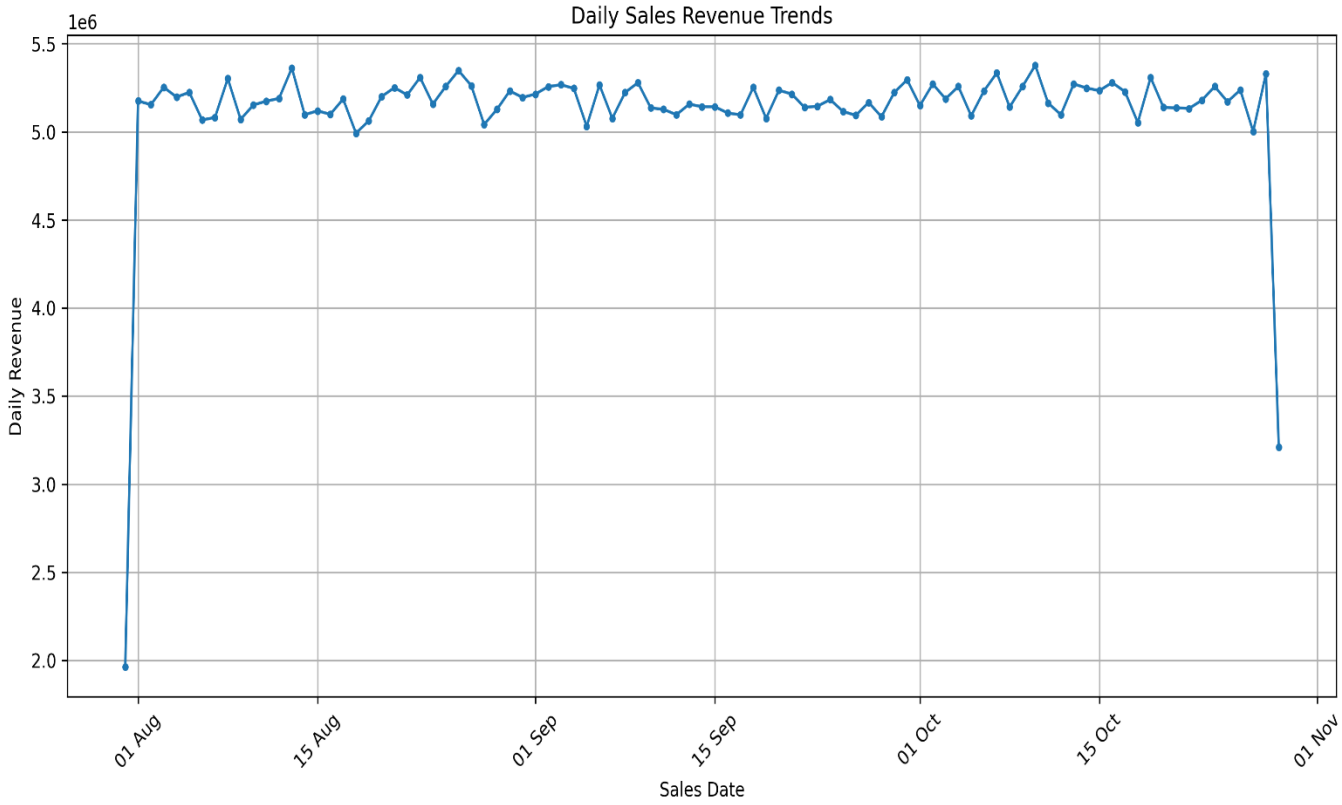
| Metric | Value |
|---|---|
| **Total revenue** | 466,576,163.22 |
| **Average daily revenue** | 5,127,210.58 |
| **Highest daily revenue** | 5,377,908.19 |
| **Lowest daily revenue** | 1,964,275.38 |
| **Days analyzed** | 91 |

The first and last dates were partial days, so their revenue should not be compared directly with full-day values.

**Interpretation:** Daily revenue analysis provides information about sales performance over time and supports business monitoring.

**Recommendation:** Monitor daily revenue and use historical sales information to improve inventory and operational planning.

Daily Sales Trends sales trends.




## 8.7 Summary of Visualization Findings

| Business area | Main finding | Recommendation |
|---|---|---|
| **Customer segmentation** | Top 25% generate 30.57% of revenue | Retain high-value customers |
| **Product views** | prod_02607 has 2,553 views | Monitor customer interest |
| **Product purchases** | prod_00283 sold 661 units | Maintain availability of strong sellers |
| **Conversion funnel** | Largest drop is 46.95% from home to product detail | Improve product discovery |
| **Sales trends** | 466.58 million revenue over 91 dates | Monitor sales performance |

# 9. SCALABILITY AND TECHNOLOGY SELECTION

### 9.1 MongoDB Scalability

MongoDB supports horizontal scaling through sharding.

As the number of customers, products, transactions, and sessions increases, data can be distributed across multiple shards.

Appropriate shard keys and indexes are important to maintain balanced storage and efficient queries.

## 9.2 HBase Scalability

HBase is designed for distributed storage and horizontal scaling.

Additional Region Servers can be introduced to distribute table regions and increase storage and processing capacity.

A good row-key design is essential because poorly selected keys can create uneven access patterns and hotspots.

## 9.3 Spark Scalability

Spark supports distributed processing across multiple worker nodes.

The project used local execution for development. A larger implementation could use a cluster with multiple executors and greater memory resources.

Partitioning, caching, efficient file formats, and avoiding unnecessary shuffles would help improve performance.

## 9.4 Technology Selection Summary

| Analytical requirement | Selected technology | Reason |
|---|---|---|
| **Flexible customer profiles** | MongoDB | Document structure |
| **Nested transaction items** | MongoDB | Embedded arrays |
| **Product catalog** | MongoDB | Flexible documents and queries |
| **Large browsing event storage** | HBase | Wide-column and row-key access |
| **Time-series activity retrieval** | HBase | Efficient key-range scans |
| **Large-scale transformations** | Spark | DataFrame and distributed processing |
| **Complex aggregations** | Spark SQL | Analytical query capabilities |
| **Business visualizations** | Matplotlib | Static analytical graphs |

# 10. CHALLENGES AND LIMITATIONS

## 10.1 Hardware and Resource Constraints

The project was developed on a local Windows computer with limited memory resources.

Spark was therefore configured using local [2] to reduce resource consumption.

## 10.2 Synthetic Data Limitations

The project used synthetic data rather than real e-commerce customer records.

The results therefore demonstrate analytical methods and simulated business patterns rather than actual market behavior.

# 12. CONCLUSION

This project demonstrated the strategic application of MongoDB, Apache HBase, and Apache Spark in an e-commerce Big Data analytics environment.

The generated dataset contained 10,000 users, 5,000 products, 25 categories, 500,000 transactions, and 2,000,000 browsing sessions.

MongoDB provided flexible document storage, indexing, validation, and analytical querying. HBase demonstrated the wide-column data model and its suitability for large-scale and time-series information. Apache Spark provided data processing capabilities through DataFrames, SQL analytics, cohort analysis, recommendation indicators, and batch processing.

The analytical results identified valuable customers, high-performing products, customer conversion challenges, and daily revenue patterns.

Five visualizations were generated to communicate these findings and support business recommendations.

The project also highlighted important technology trade-offs. MongoDB is effective for flexible and nested document data, HBase is suitable for large-scale row-key-oriented access, and Spark is useful for transforming and analyzing large datasets.

Although limitations were encountered during HBase ingestion and full cross-database connector integration, the project demonstrates the complementary roles of the three technologies and provides a foundation for a more scalable e-commerce analytics solution.

APPENDIX:

LIST OF 25 PRODUCTS CATEGORIES:

These are the category identifiers and their corresponding names in dataset.

| No. | Category ID | Category name |
|---|---|---|
| **1** | cat_000 | Rodriguez, Figueroa and Sanchez |
| **2** | cat_001 | Baldwin Ltd |
| **3** | cat_002 | James LLC |
| **4** | cat_003 | Bush Inc |
| **5** | cat_004 | Kelly-Santiago |
| **6** | cat_005 | Cameron-Parsons |
| **7** | cat_006 | Ellis, Baker and Wright |
| **8** | cat_007 | Moore, Becker and Carlson |
| **9** | cat_008 | Rodriguez-Graham |
| **10** | cat_009 | Skinner-Romero |
| **11** | cat_010 | Burns-Rodriguez |
| **12** | cat_011 | Spence PLC |
| **13** | cat_012 | Carney-Santos |
| **14** | cat_013 | Smith Ltd |
| **15** | cat_014 | Anderson, Walls and Duncan |
| **16** | cat_015 | Owens-Daniel |
| **17** | cat_016 | Wright-Leblanc |
| **18** | cat_017 | Hancock Inc |
| **19** | cat_018 | Phillips Inc |
| **20** | cat_019 | Powell PLC |
| **21** | cat_020 | Brewer, Bryant and Hickman |
| **22** | cat_021 | Mitchell-Kim |
| **23** | cat_022 | Preston Ltd |
| **24** | cat_023 | King-Odonnell |
| **25** | cat_024 | Patterson, Smith and Jones |

For example, of product name code

| Code / Identifier | What it represents | Corresponding name |
|---|---|---|
| **`user_005142`** | Customer ID | Actual customer's name |
| **`prod_00283`** | Product ID | Actual product name |
| **`cat_005`** | Category ID | Cameron-Parsons |
| **`txn_...`** | Transaction ID | Individual purchase transaction |
| **`session_...`** | Session ID | Individual website visit/session |

The prefixes mean:

- `user_` = User/customer.
- `prod_` = Product.
- `cat_` = Product category.
- `txn_` = Transaction.
- `session_` = Website session.